

Búsqueda no informada

Para este ejercicio elegí la pareja Fagaras → Lugoj. Como se puede ver en la Figura 1.

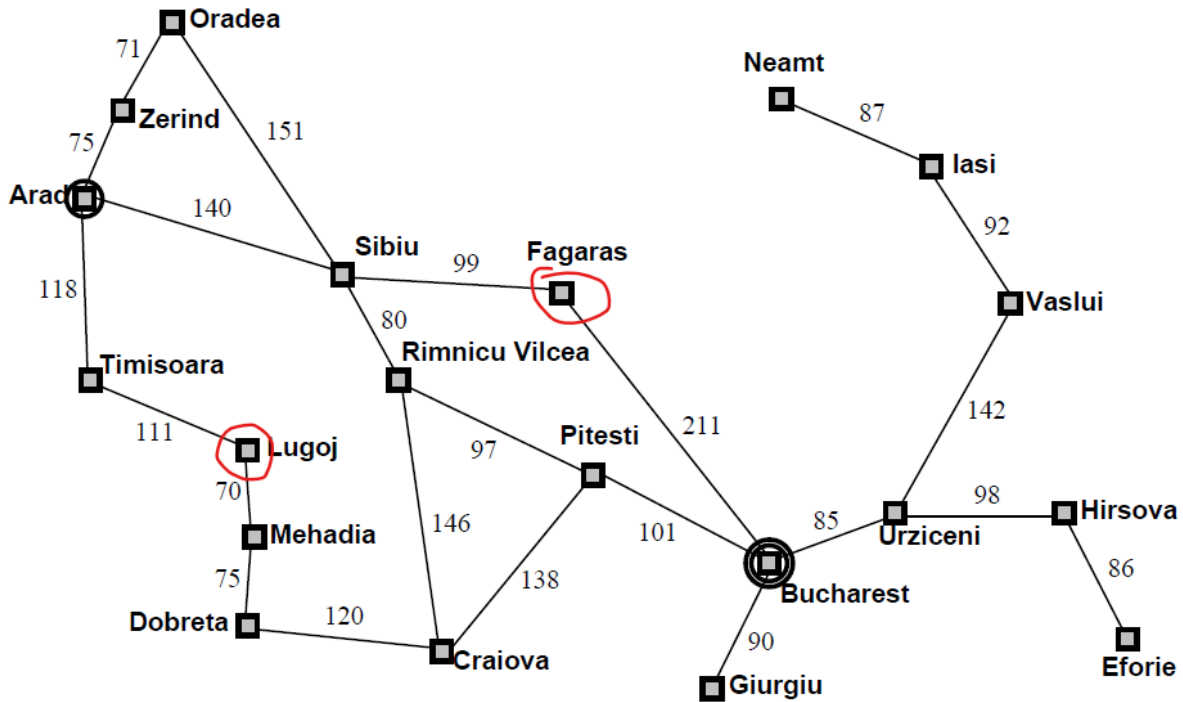


Figura 1. Grafo nuevo.

Una vez ejecutados los 5 algoritmos diferentes, se obtuvieron los siguientes resultados, ver Tabla 1.

Algoritmo	Status	Path	Depth (roads)	Cost (km)	Expanded	Generated
BFS	success	Fagaras → Sibiu → Arad → Timisoara → Lugoj	4	468	13	35
UCS	success	Fagaras → Sibiu → Arad → Timisoara → Lugoj	4	468	15	39
DFS	success	Fagaras → Bucharest → Pitesti → Craiova → Drobeta → Mehadia → Lugoj	6	715	7	18
DLS (Límite 3)	cutoff	Ninguno (Límite alcanzado)	-	-	9	26
DLS (Límite 4)	success	Fagaras → Sibiu → Arad → Timisoara → Lugoj	4	468	12	29
IDS	success	Fagaras → Sibiu → Arad → Timisoara → Lugoj	4	468	25	70

Tabla 1. Tabla comparativa entre resultados.

Contestando la primera pregunta, efectivamente, el algoritmo BFS encontró el camino con menos saltos entre carreteras, usando un total de 4 carreteras como se puede ver en la Tabla 1. Y el algoritmo UCS, enfocada precisamente en encontrar el camino con menor recorrido, utilizó el mismo camino que BFS, ya que en este caso el camino más corto es el camino con menos saltos entre carreteras.

Donde se puede observar una diferencia entre rutas, fue en el algoritmo DFS, que justo como menciona la actividad suele usar un camino más largo, ya sea en kilómetros o carreteras, esto sucede debido a que no tiene noción de optimalidad, ni de costo, el objetivo de este algoritmo es explorar un camino hasta llegar a la meta o fallar (lo que suceda primero). Si por casualidad la primera rama que elige tomar resulta ser un recorrido que zigzaguea por todo el mapa antes de conectar finalmente con el destino, DFS lo aceptará como la solución válida de inmediato y terminará la ejecución.

Ahora bien, para el algoritmo donde podemos usar un límite de caminos, el cutoff sucede hasta el límite 3, donde si usamos un límite de 4, entonces devuelve la solución exacta que el algoritmo BFS. Como BFS expande por niveles, la profundidad a la que BFS encuentra el objetivo es exactamente la misma que el límite en el que DLS logra llegar a él sin sufrir un corte. DS no es más que un bucle que ejecuta DLS aumentando el límite gradualmente. Por esta razón, IDS simula el comportamiento de exploración por niveles de BFS usando la eficiencia de memoria de DFS. IDS encontrará su solución en la misma iteración (Límite = N) que equivale a la profundidad donde BFS halla el camino con menos carreteras.

Evidencia de ejecución

```
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestria IA\inteligencia-artificial\Búsqueda no informada\project> python 02_breadth_first_search.py --from-city Fagaras --to Lugoj
Algorithm: Breadth-first search
Problem: Fagaras → Lugoj
Status: success
Path: Fagaras → Sibiu → Arad → Timisoara → Lugoj
Depth: 4 roads
Cost: 468 km
Expanded: 13 nodes
Generated: 35 nodes
Frontier: max size 7
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestria IA\inteligencia-artificial\Búsqueda no informada\project> python 03_uniform_cost_search.py --from-city Fagaras --to Lugoj
Algorithm: Uniform-cost search
Problem: Fagaras → Lugoj
Status: success
Path: Fagaras → Sibiu → Arad → Timisoara → Lugoj
Depth: 4 roads
Cost: 468 km
Expanded: 15 nodes
Generated: 39 nodes
Frontier: max size 7
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestria IA\inteligencia-artificial\Búsqueda no informada\project> python 04_depth_first_search.py --from-city Fagaras --to Lugoj
Algorithm: Depth-first search
Problem: Fagaras → Lugoj
Status: success
Path: Fagaras → Bucharest → Pitesti → Craiova → Drobeta → Mehadia → Lugoj
Depth: 6 roads
Cost: 715 km
Expanded: 7 nodes
Generated: 18 nodes
Frontier: max size 4
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestria IA\inteligencia-artificial\Búsqueda no informada\project> python 05_depth_limited_search.py --limit 1 --from-city Fagaras --to Lugoj
Algorithm: Depth-limited search
Problem: Fagaras → Lugoj
Status: cutoff
Detail: limit=1
Expanded: 1 nodes
Generated: 3 nodes
Frontier: max size 3
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestria IA\inteligencia-artificial\Búsqueda no informada\project> python 05_depth_limited_search.py --limit 2 --from-city Fagaras --to Lugoj
Algorithm: Depth-limited search
Problem: Fagaras → Lugoj
Status: cutoff
Detail: limit=2
Expanded: 3 nodes
Generated: 11 nodes
Frontier: max size 6
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestria IA\inteligencia-artificial\Búsqueda no informada\project> python 05_depth_limited_search.py --limit 3 --from-city Fagaras --to Lugoj
Algorithm: Depth-limited search
Problem: Fagaras → Lugoj
Status: cutoff
Detail: limit=3
Expanded: 9 nodes
Generated: 26 nodes
Frontier: max size 6
```

Elaborado por: César Ramayo

```
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestría IA\inteligencia-artificial\Búsqueda no informada\project> python 05_depth_limited_search.py --limit 4 --from-city Fagaras --to Lugoj
Algorithm: Depth-limited search
Problem: Fagaras → Lugoj
Status: success
Detail: limit=4
Path: Fagaras → Sibiu → Arad → Timisoara → Lugoj
Depth: 4 roads
Cost: 468 km
Expanded: 12 nodes
Generated: 29 nodes
Frontier: max size 7
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestría IA\inteligencia-artificial\Búsqueda no informada\project> python 06_iterative_deepening_search.py --from-city Fagaras --to Lugoj
Algorithm: Iterative deepening search
Problem: Fagaras → Lugoj
Status: success
Detail: last_limit=4
Path: Fagaras → Sibiu → Arad → Timisoara → Lugoj
Depth: 4 roads
Cost: 468 km
Expanded: 25 nodes
Generated: 70 nodes
Frontier: max size 7
```

Reto opcional

Para este reto opcional, usaré la pareja: Sibiu → Bucharest.

```
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestría IA\inteligencia-artificial\Búsqueda no informada\project> python 02_breadth_first_search.py --from-city Sibiu --to Bucharest
Algorithm: Breadth-first search
Problem: Sibiu → Bucharest
Status: success
Path: Sibiu → Fagaras → Bucharest
Depth: 2 roads
Cost: 310 km
Expanded: 3 nodes
Generated: 9 nodes
Frontier: max size 5
• (.venv) PS C:\Users\Cesar Ramayo\Documents\Maestría IA\inteligencia-artificial\Búsqueda no informada\project> python 03_uniform_cost_search.py --from-city Sibiu --to Bucharest
Algorithm: Uniform-cost search
Problem: Sibiu → Bucharest
Status: success
Path: Sibiu → Rimnicu Vilcea → Pitesti → Bucharest
Depth: 3 roads
Cost: 278 km
Expanded: 9 nodes
Generated: 25 nodes
Frontier: max size 6
```

En la figura de arriba, se puede observar que el algoritmo BFS, utiliza 2 caminos para llegar al destino, recorriendo en total 310km, expandió 3 nodos, y generó 9 nodos. En comparación con el algoritmo UCS, donde encontró un camino más corto de un total de 278km, aunque usando una carretera más, siendo un total de 3 carreteras, y expandió 9 nodos y generó 25 nodos. Entonces, podemos decir que el algoritmo de costo trabajó más, por lo que entendí esto sucede debido a que el algoritmo 02 utiliza un “Early Goal Test”, es decir, que revisa si un nodo es el objetivo en el instante exacto en que lo genera (descubriéndolo al mirar los vecinos).

1. Expandió Sibiu.
2. Expandió Arad (o el nodo que seguía por orden alfabético).
3. Expandió Fagaras, entonces ve Bucharest como vecino.
4. Como la prueba de meta es temprana, se detiene ahí, y entonces declara que éxito, al hallar Bucharest.

En comparación con UCS, donde si halla el mismo camino que BFS, pero mete este camino a la cola, entonces busca un camino alternativo, y expande cualquier otro nodo con menor kilometraje. Entonces revisa ciudades como Rimnicu Vilcea (80 km), Arad (140 km), Oradea (151 km), Pitesti (177 km) y Craiova (226 km). Solo cuando la ruta a través de Pitesti sumó 278 km, superó en prioridad a la ruta de 310 km, se colocó al frente de la cola, y finalmente fue expandida.